PROJECT REQUIREMENTS SPECIFICATION

Graduation Internship Project | 22 Weeks

Multi-Agent PentestAI & Mitigation Platform

for ComunikCRM

Field	Detail
Company	ComunikCRM
Target Environment	Linux Infrastructure (VoIP/SIP, Web, DB, DNS, SMTP, and any other exposed services)
Attacker Machine	Parrot OS (local machine)
Total Duration	22 weeks | Phase 1: 18 weeks | Phase 2: 4 weeks
Agent Framework	CrewAI / LangChain (Python)
LLM Runtime	Ollama Qwen2.5:8B / DeepSeek-R1:8B (local inference)
Memory	ChromaDB (vector) + JSON state (persistent)
Knowledge Base	Exploit-DB RAG, MITRE ATT&CK, CVE feeds, GTFOBins, PayloadsAllTheThings
Infra Automation	Ansible (Phase 2: pentest dispatch + mitigation deployment)

Table of Contents

1. Context and Scope
2. Problem Statement
3. Proposed Solution
4. Phase 1 Architecture: Multi-Agent Attack Chain
5. Phase 2 Architecture: Ansible Integration and Remediation
6. Detailed Agent Specifications
7. Technical Stack
8. Knowledge Base and RAG Pipeline
9. Development Plan: 22 Weeks
10. Test Environments
11. Deliverables
12. Success Indicators (KPIs)
13. Constraints and Risks
14. Conclusion

1. Context and Scope

1.1 General Context

ComunikCRM operates a fully Linux-based infrastructure hosting a variety of critical services: VoIP/SIP telephony (Asterisk, PBX), application servers (Apache, PHP, MySQL, MongoDB), DNS, SMTP/mail services, REST APIs, and any other services that may be present or added over time. Regular and thorough security assessment of this environment is essential but currently performed manually, at high cost, and at low frequency.

This project aims to design, build, and validate an autonomous multi-agent AI platform, operating from a Parrot OS attacker machine, capable of conducting a complete penetration test against a target given only as a domain name. In Phase 2, the platform integrates with the company's infrastructure via Ansible to deliver findings to an admin dashboard and apply approved mitigations on the victim machine.

1.2 Project Phases

Phase	Scope	Duration
Phase 1	Autonomous multi-agent pentest platform: validated on Metasploitable2, then on a ComunikCRM test server	18 weeks
Phase 2	Ansible integration, simplified reporting to Admin Dashboard, Mitigation Agent deployment on victim machine, confirmation email to admin	4 weeks

1.3 Targets

- Phase 1A: Metasploitable2 (local lab): full platform validation across all agents
- Phase 1B: ComunikCRM test server (services TBD: may include Apache, DNS, VoIP/SIP, SMTP, Web, DB, or any combination)
- Phase 2: ComunikCRM real infrastructure, accessed via Ansible using only a domain name as input

2. Problem Statement

The following table summarizes the key problems and their operational impact:

Problem	Impact
High cost of manual pentests	External missions cost thousands of euros and take 1 to 3 weeks
Low test frequency	Security gaps remain undetected between audits
Automated scanners lack reasoning	No exploit chaining, high false positive rate, generic reports
No contextual memory between scans	Each scan starts from scratch without leveraging prior discoveries
No remediation loop	Long delay between vulnerability detection and actual fix applied
Partial attack surface coverage	Manual tests are time-boxed and may miss services based on available scope

What is missing is a solution capable of reasoning like a human expert, autonomously chaining attack steps, adapting to any discovered services, and closing the security loop with a human-approved remediation pipeline.

3. Proposed Solution

The Multi-Agent PentestAI and Mitigation Platform is a Python-based multi-agent AI system built on CrewAI / LangChain, powered by locally running LLMs (Ollama + Qwen2.5:8B / DeepSeek-R1:8B), and enriched by a RAG knowledge base indexed in ChromaDB.

Given only a domain name, the platform:

- Discovers the entire attack surface automatically (DNS, subdomains, ports, services, banners)
- Follows the full attack chain: Recon, Enumeration, Vulnerability Scan, Exploitation, Privilege Escalation, Post-Exploitation
- Adapts dynamically to whatever services are running on the target with no hardcoded assumptions
- Chains exploits intelligently based on prior findings, using MITRE ATT&CK as a reasoning backbone
- Generates a full professional report with CVSS scoring and MITRE ATT&CK mapping (Phase 1)
- Delivers a simplified admin-readable findings report and mitigation action list via Ansible/email (Phase 2)
- Deploys a Mitigation Agent on the victim machine upon admin approval, then confirms execution by notification to the dashboard

4. Phase 1 Architecture: Multi-Agent Attack Chain

The diagram below illustrates the full multi-agent attack chain running on the Parrot OS attacker machine. The Orchestrator Agent receives the target domain name and drives all specialized agents through the attack chain. Each agent spawns parallel threads whose number and type adapt dynamically to what is discovered on the target.

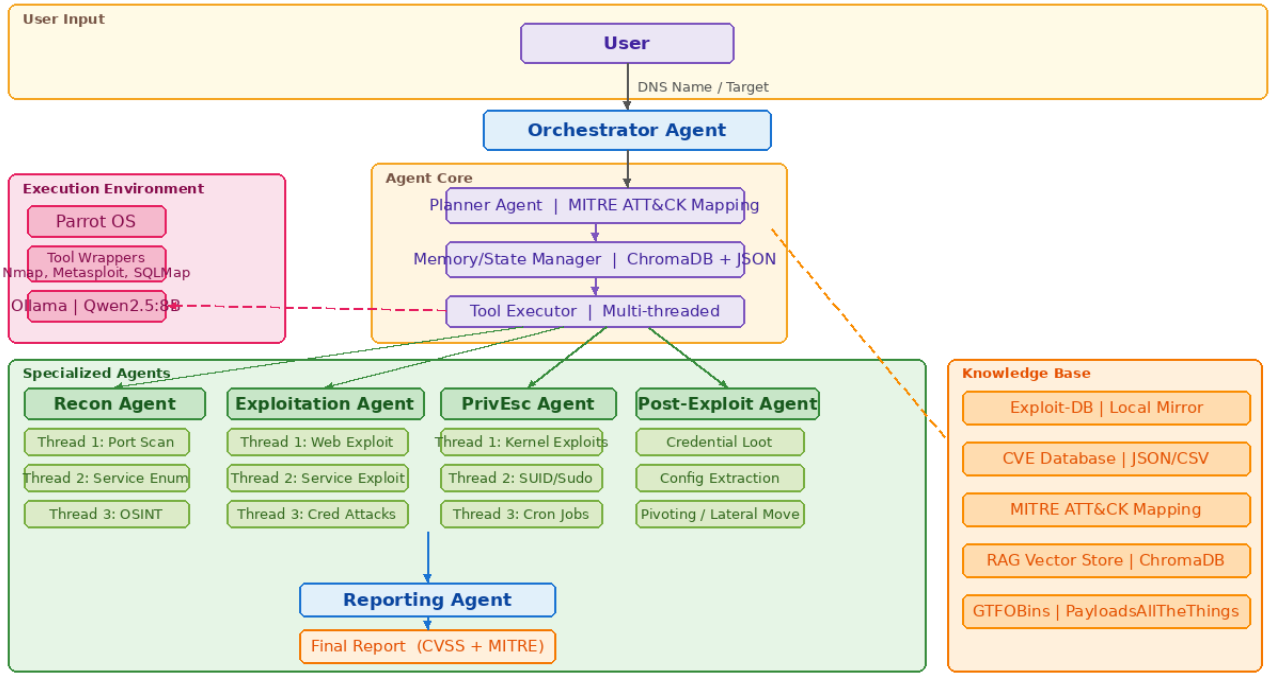

Figure 1 — Phase 1: Multi-Agent Attack Chain Architecture

4.1 Core ReAct Loop

Every agent in the system follows the ReAct paradigm (Reasoning + Acting):

Step	Description
Thought	The agent reasons about the current state, prior observations, and the next best action using the LLM
Action	The agent calls a tool (nmap, Metasploit, SQLMap, etc.) or delegates to a sub-agent
Observation	The output is parsed, structured, stored in ChromaDB, and fed back into the next reasoning cycle

4.2 Attack Chain Flow

The Orchestrator Agent enforces the following sequential attack chain, passing enriched context at each handoff:

#	Agent	Responsibilities	Output to Next Agent
1	Recon Agent	DNS, subdomain discovery, OSINT, port scanning (parallel threads)	Host list, open ports, service banners
2	Enumeration Agent	Deep service fingerprinting, VoIP enum, web directory brute-force, banner grabbing	Service versions, exposed endpoints, tech stack
3	Vuln Scan Agent	CVE matching against discovered services, RAG exploit search, CVSS scoring, target prioritization	Ranked vulnerability list with exploitability score
4	Exploitation Agent	Controlled exploit execution per vector (Web, Service, Credentials) in parallel threads	Shell access, extracted data, confirmed exploits
5	PrivEsc Agent	Kernel exploits, SUID/Sudo abuse, misconfiguration hunting (parallel threads)	Root or elevated shell, PrivEsc path documented
6	Post-Exploit Agent	Credential loot, config file extraction, lateral movement and pivoting	Credential dump, internal hosts, sensitive data
7	Reporting Agent	CVSS scoring, MITRE ATT&CK tagging, full technical + executive report generation	Final pentest report (PDF / terminal)

5. Phase 2 Architecture: Ansible Integration and Remediation

Phase 2 connects the attacker machine to the ComunikCRM infrastructure via Ansible and introduces a full Human-in-the-Loop remediation loop. The admin dashboard is the central approval gate. No mitigation is applied without explicit admin approval.

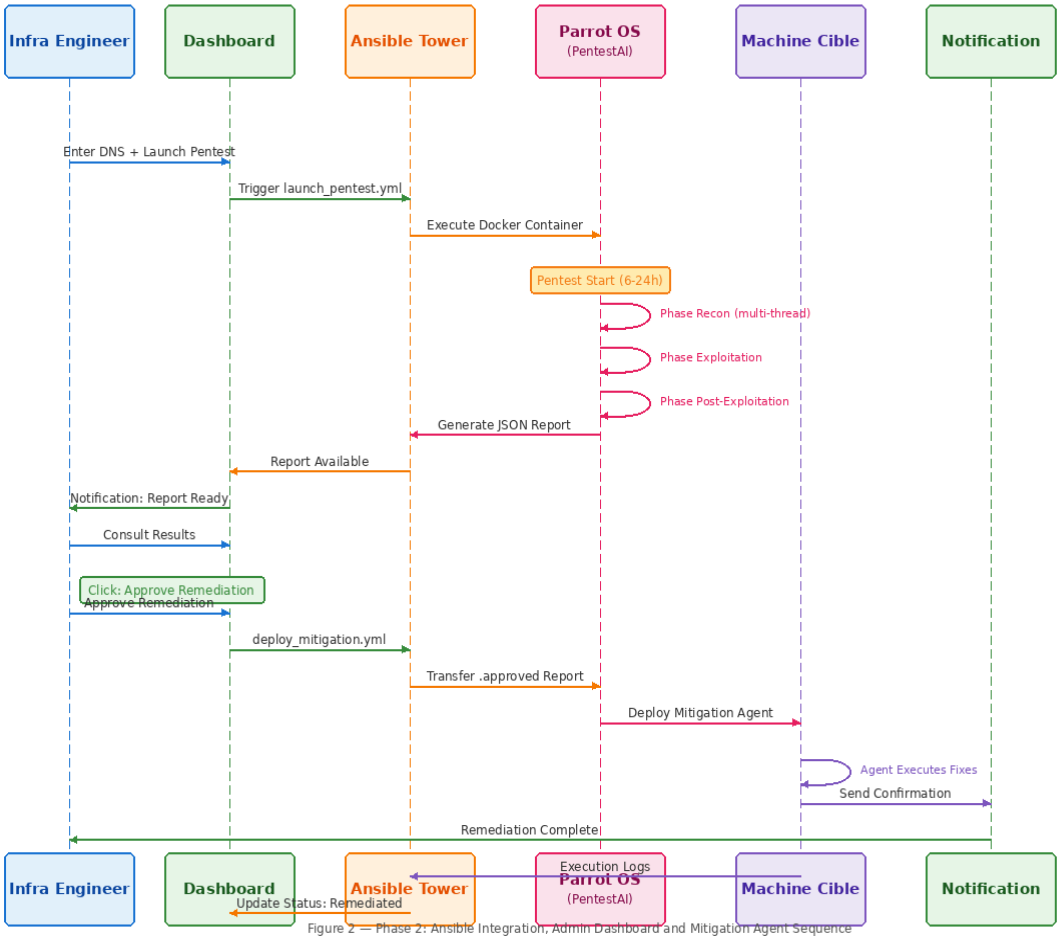

Figure 2 — Phase 2: Ansible Integration, Admin Dashboard and Mitigation Agent Sequence

5.1 Phase 2 Workflow

Step	Actor	Action
1	Infra Engineer	Enters a domain name in the Dashboard and clicks Launch Pentest
2	Ansible Tower	Triggers launch_pentest.yml, which executes the PentestAI Docker container on Parrot OS
3	Parrot OS (PentestAI)	Full attack chain runs autonomously (6 to 24 hours max depending on target complexity)
4	Parrot OS (PentestAI)	Reporting Agent generates JSON report: critical findings + prioritized mitigation action list
5	Ansible Tower	Report sent back to Dashboard; notification sent to admin
6	Infra Engineer	Reviews findings on Dashboard, approves or rejects each mitigation action (Human-in-the-Loop)

7	Ansible Tower	Triggers deploy_mitigation.yml: deploys Mitigation Agent to victim machine with approved actions only
8	Mitigation Agent	Executes approved actions on victim machine (patch, harden, disable, fix configs)
9	Mitigation Agent	Sends confirmation notification to admin + pushes execution logs back to Dashboard

Critical constraint: the Mitigation Agent only executes what is explicitly present in the approved list. It has no autonomous decision authority beyond those specific approved actions.

6. Detailed Agent Specifications

6.1 Orchestrator Agent

The central controller of the platform. It is the only agent that receives the user input (domain name). It drives the full attack chain by delegating to specialized agents in order, passing enriched context at each step, and maintaining the global mission state in ChromaDB and JSON.

Attribute	Detail
Framework	CrewAI Manager LLM or LangChain AgentExecutor
LLM	Ollama local: Qwen2.5:8B (default) or DeepSeek-R1:8B (deep reasoning tasks)
Memory	ChromaDB (vector semantic search) + JSON state file (persistent mission state)
Input	Domain name (e.g. target.comunikcrm.local)
Output	Final pentest report + mission state log

6.2 Recon Agent

Handles the reconnaissance phase. Spawns parallel threads whose number and purpose adapt to what is discovered. Example thread types:

- Thread 1: DNS and subdomain enumeration using subfinder, amass, dnsrecon, dnsenum
- Thread 2: OSINT and header analysis using theHarvester, WhatWeb, Whois
- Thread 3: Active network recon using nmap, ping sweep, traceroute

Results are merged and indexed into ChromaDB for use by downstream agents.

6.3 Enumeration Agent

Deepens service discovery based on Recon output. Adapts its enumeration strategy to the services found:

- Nmap NSE scripts for deep version detection and service fingerprinting
- SIPVicious (svmap, svwar) for VoIP/SIP surface enumeration
- gobuster or ffuf for web directory, vhost, and endpoint brute-force
- Banner grabbing for SSH, SMTP, FTP, and other TCP services

6.4 Vulnerability Scan Agent

Correlates discovered service versions with known vulnerabilities:

- RAG query against local Exploit-DB mirror and CVE database for each service/version pair
- Automated CVSS scoring per finding
- Searchsploit CLI for offline exploit lookup
- Nikto for web surface scanning
- Produces a ranked and scored vulnerability list as input to the Exploitation Agent

6.5 Exploitation Agent

Executes controlled exploits in parallel threads, one per attack vector. Actual vectors depend on what is discovered. The agent adapts dynamically:

Vector (example)	Example Tools	Example Attacks
SSH / Linux services	Hydra, Metasploit, paramiko	Brute force, auth bypass (CVE-based)
Web / PHP / APIs	SQLMap, wfuzz, Metasploit web modules	SQLi, LFI, RCE, file upload bypass
VoIP / SIP	SIPVicious, Metasploit SIP modules	SIP enumeration, registration attacks, eavesdrop
SMTP / Mail	swaks, smtp-user-enum	User enumeration, open relay, spoofing
Databases (MySQL/Mongo/...)	SQLMap, nmap NSE DB scripts	Unauthenticated access, credential dump
DNS	dnsrecon, fierce	Zone transfer, cache poisoning
Any other service discovered	Searchsploit + RAG lookup	CVE-matched exploit from Exploit-DB

6.6 Privilege Escalation Agent

Activated after a shell or initial access is confirmed. Runs parallel discovery threads to find escalation paths:

- Thread 1: Kernel exploit search using local CVE database and Exploit-DB RAG
- Thread 2: SUID/Sudo misconfiguration hunting using GTFOBins knowledge base
- Thread 3: Cron jobs, writable paths, world-writable files, and service misconfigurations
- Primary tools: LinPEAS, GTFOBins (RAG-indexed)

6.7 Post-Exploitation Agent

Operates after privilege escalation is confirmed:

- Credential extraction: /etc/shadow, .bash_history, config files, database credentials
- Internal network discovery for pivoting to additional hosts
- Documentation of persistence paths (documented only, not executed in production scope)

6.8 Reporting Agent

Produces two report types depending on the phase:

- Phase 1 full report: detailed findings, proof-of-concept outputs, CVSS scores, MITRE ATT&CK technique mapping, executive summary and technical appendix
- Phase 2 simplified report: critical findings only, non-technical language, ordered mitigation action list ready for admin review

6.9 Mitigation Agent (Phase 2 only)

Deployed onto the victim machine via Ansible after admin approval:

- Receives the exact list of approved mitigation actions and only those
- Applies patches, closes ports, hardens service configurations, disables unnecessary services
- Generates an execution confirmation report
- Sends a confirmation notification to the admin

> *The Mitigation Agent has no autonomous decision authority. It executes only the actions explicitly listed in the approved report. Nothing more.*

7. Technical Stack

Category	Technology	Role
Language	Python 3.11+	All agents, tools, pipelines
Agent Framework	CrewAI (primary) / LangChain (fallback)	Multi-agent orchestration, ReAct loop
LLM Runtime	Ollama + Qwen2.5:8B / DeepSeek-R1:8B	Local inference, reasoning, planning
Embedding Model	nomic-embed-text (via Ollama)	Vector embeddings for ChromaDB RAG
Vector Memory	ChromaDB	Semantic search, RAG, persistent context
State Persistence	JSON files + SQLite	Mission state, logs, findings across agents
Attacker OS	Parrot OS	Pre-installed offensive tools, attack platform
Pentest Tools	nmap, Metasploit, Hydra, SQLMap, SIPVicious, gobuster, ffuf, nikto, amass, subfinder, dnsrecon, searchsploit, swaks, LinPEAS...	Pentest execution per attack vector
Parallelism	Python threading / asyncio	Concurrent threads per agent
Infra Automation	Ansible	Pentest dispatch + Mitigation Agent deployment (Phase 2)
Notification	Python smtplib (SMTP)	Admin email: findings report + confirmation
MITRE Mapping	MITRE ATT&CK JSON (local)	Technique tagging in reports
Knowledge Base	Exploit-DB + CVE feeds + GTFOBins	RAG-indexed exploit and technique database

> *All LLM inference runs locally via Ollama. No external API calls. No data leaves the machine. If fine-tuning a specialized pentest model is needed, Kaggle Notebooks or Google Colab can be used for training, with the resulting model exported and served locally via Ollama.*

8. Knowledge Base and RAG Pipeline

8.1 Indexed Sources

Source	Content	Usage
Exploit-DB (local mirror)	CVEs, exploits, PoC code	RAG: find exploits matching service/version
MITRE ATT&CK (JSON)	Tactics, techniques, procedures	Automatic technique tagging in reports
NVD / CVE feeds	Official CVE database	CVSS scoring, vulnerability enrichment
GTFOBins	Exploitable Linux binaries	PrivEsc agent: sudo/SUID abuse paths
PayloadsAllTheThings	Web payloads, reverse shells	Exploitation agent enrichment
GitLab / internal docs (optional)	Company-specific tech stack docs	Context-aware attack adaptation

8.2 RAG Pipeline

1. Ingestion: sources are parsed, chunked, and indexed into ChromaDB using nomic-embed-text embeddings
2. Retrieval: when a service/version is identified, the agent runs a semantic query to retrieve the most relevant exploits and techniques
3. Augmented Generation: the LLM receives the RAG context alongside the current mission state to plan the next action

9. Development Plan: 22 Weeks

Weeks	Phase	Tasks	Deliverable
S1-S2	Foundations	Parrot OS setup, Ollama + ChromaDB + CrewAI installation, architecture design, Exploit-DB RAG ingestion	Functional environment + RAG operational
S3-S4	Orchestrator	Orchestrator Agent, ReAct engine, JSON state manager, inter-agent communication protocol	Orchestrator operational
S5-S6	Recon	Recon Agent with parallel threads: DNS, OSINT, active recon. Integration: subfinder, amass, nmap, dnsrecon	Recon Agent validated
S7-S8	Enum + VulnScan	Enumeration Agent (SIPVicious, gobuster, nikto, Nmap NSE). VulnScan Agent with CVE matching and CVSS scoring	Enum + VulnScan validated
S9-S11	Exploitation	Exploitation Agent with parallel threads per vector: Web, Service, Credentials. Exploit chaining logic	Exploitation Agent validated
S12-S13	PrivEsc + Post-Exploit	PrivEsc Agent: LinPEAS, GTFOBins, SUID/Sudo. Post-Exploit Agent: loot, credential extraction, pivot	PrivEsc + Post-Exploit validated
S14-S15	Reporting	Reporting Agent Phase 1 (full report: CVSS + MITRE ATT&CK). Reporting Agent Phase 2 (simplified + mitigation list)	Reporting Agent validated
S16-S17	Phase 1 Testing	Full pentest run on Metasploitable2 then ComunikCRM test server. Bug fixes and tuning	Phase 1 pentest reports
S18	Phase 1 Wrap-up	Benchmarks vs manual pentest, performance optimization, Phase 1 technical documentation	Phase 1 complete
S19-S20	Phase 2 Ansible	Ansible pentest playbook integration, Mitigation Agent development,	Full Ansible pipeline
S21	Phase 2 Testing	End-to-end Phase 2 test: pentest trigger, admin approval, mitigation,notif confirmation	Phase 2 validated
S22	Final Defense	Full Phase 1 and Phase 2 scenario demonstrated end-to-end. PFE report writing and final presentation	PFE Report + Defense

10. Test Environments

10.1 Phase 1A: Metasploitable2 (Local Lab)

- Metasploitable2 VM on an isolated network (VirtualBox/VMware on attacker machine)
- Full attack chain validation: Recon, Enum, VulnScan, Exploit, PrivEsc, Post-Exploit, Report
- Success criterion: root shell obtained and full report generated in under 6 hours

10.2 Phase 1B: ComunikCRM Test Server

- Server provided by ComunikCRM with real running services (exact services TBD: may include any combination of DNS, VoIP/SIP, SMTP, Web, DB, APIs)
- Non-destructive pentest within an authorized and contractually defined scope
- Success criterion: critical findings detected and professional report generated automatically

10.3 Phase 2: ComunikCRM Real Infrastructure

- Access provided by the infrastructure engineer at ComunikCRM via Ansible
- Single input to the platform: domain name only
- Output: admin dashboard notification + simplified email report
- Remediation applied on victim machine only after explicit admin approval

11. Deliverables

#	Deliverable	Week	Description
L1	PentestAI Platform source code	S22	All agents, tool wrappers, configurations, RAG pipeline
L2	Metasploitable2 pentest report	S17	Auto-generated report by the platform (Phase 1A)
L3	ComunikCRM test server pentest report	S18	Professional report from Phase 1B
L4	Ansible Phase 2 playbooks	S20	Pentest dispatch + Mitigation Agent deployment playbooks
L5	Technical documentation	S22	Architecture, installation guide, usage manual
L6	PFE thesis report	S22	Full written academic report
L7	Final defense presentation	S22	Slides + live end-to-end demonstration

12. Success Indicators (KPIs)

Indicator	Target	Phase
Full pentest on Metasploitable2 completed	< 6 hours	Phase 1A
Vulnerability detection rate (known vulns in lab)	>= 85%	Phase 1
False positive rate	<= 10%	Phase 1
Automated multi-step exploit chaining	Validated in lab	Phase 1
CVSS + MITRE ATT&CK report auto-generated	100%	Phase 1
Simplified findings report delivered to admin	< 5 min post-pentest	Phase 2
Mitigations successfully applied after approval	>= 80%	Phase 2
Admin confirmation notif sent after remediation	100%	Phase 2

13. Constraints and Risks

13.1 Constraints

- Legal and ethical: all penetration tests are performed exclusively on authorized environments. A written authorization agreement with ComunikCRM is mandatory before Phase 1B and Phase 2 begin.
- Local compute: all LLM inference runs locally on the attacker machine via Ollama. Minimum recommended hardware: 16 GB RAM, modern CPU or GPU for faster inference.
- No UI in Phase 1: the platform operates entirely from the terminal in Phase 1. There is no graphical dashboard.
- Mitigation scope: the Mitigation Agent executes only the actions explicitly listed in the approved report. No autonomous decisions beyond that list.
- Dynamic target: the platform must handle any Linux service combination on the target with no hardcoded assumptions about what services are running.

13.2 Risk Table

Risk	Likelihood	Mitigation Strategy
Local LLM too slow for real-time reasoning	Medium	Use Q4 quantized Qwen2.5:8B, optimize prompts and context window usage
LLM hallucination on exploit details	High	Strict RAG grounding from local Exploit-DB; validate by actual tool execution before reporting
Ansible connectivity blocked in Phase 2	Medium	Configure Ansible master and workers
High false positives in exploitation	Medium	Confidence scoring before each exploit; optional human confirmation gate
Scope creep on agent features	Medium	Strict sprint-based development; Phase 1 and Phase 2 milestones are fixed

14. Conclusion

The Multi-Agent PentestAI and Mitigation Platform is a technically ambitious and operationally relevant project that combines state-of-the-art agentic AI with professional penetration testing methodology.

Operating from a Parrot OS attacker machine, targeting any Linux infrastructure given only a domain name, the platform autonomously follows the full attack chain from reconnaissance to post-exploitation using parallel agents, locally running LLMs, a RAG-enriched knowledge base, and MITRE ATT&CK as a reasoning framework.

Phase 1 (18 weeks) delivers a fully validated autonomous pentest platform, tested on Metasploitable2 and a real ComunikCRM server. Phase 2 (4 weeks) closes the security loop by integrating Ansible for pentest dispatch and mitigation deployment, with a strict Human-in-the-Loop approval process ensuring that no remediation action is applied without explicit admin authorization.

This project delivers direct, measurable security value to ComunikCRM while constituting a strong technical contribution to the field of autonomous offensive security.